

Autonomous Perception-Fused Driving on the Jetson Orin Nano Platform

Camera Fusion for Perception and Control — Waveshare JETANK AI Kit +
NVIDIA Jetson Orin Nano Super

Capstone Project Report

Submitted to
IMC University of Applied Sciences Krems



University of
Applied Sciences

Bachelor Programme Informatics

by

**Adam Ryan, Aidan Ryan, Denis
Vasilev, Nora Regina Cervený**

for the award of academic degree
Bachelor of Science in Engineering (BSc)

under the supervision of
Prof. Tingting Yuan

Submitted on 26.06.2026

DECLARATION OF HONOUR

We declare on our word of honour that we have written this on our own and that we have not used any sources or resources other than stated and that we have marked those passages and/or ideas that were either verbally or textually extracted from sources. This also applies to drawings, sketches, graphic representations as well as to sources from the internet. The has not been submitted in this or similar form for assessment at any other domestic or foreign post-secondary educational institution and has not been published elsewhere. The present complies with the version submitted electronically.

Adam Ryan

Aidan Ryan

Denis Vasilev

Nora Regina Cervený

ABSTRACT

This report documents a semester-long capstone project to build an autonomous, camera-fusion-driven mobile robot on a Waveshare JETANK AI Kit chassis retrofitted with an NVIDIA Jetson Orin Nano Super compute module. The project was scoped into three tasks: Task C (hardware assembly and platform integration), Task B (camera fusion for perception, combining lane detection and object detection into a single per-frame state), and Task A (camera fusion for driving, converting that fused state into safety-gated steering and throttle commands).

The team's core technical finding is that the JETANK chassis, designed around the original Jetson Nano, is not a drop-in match for the Jetson Orin Nano Super. The two platforms differ in power delivery, kernel packaging, camera driver model, I2C bus numbering, network interface naming, and motor-driver channel mapping.

Across the semester, ten distinct system-level integration issues were identified and resolved, spanning OS imaging, WiFi driver support, desktop environment, Python packaging, CSI camera enablement, OLED display, and serial servo calibration. Two blockers were not resolved before the project deadline: a power architecture mismatch requiring an independently sourced power bank, and unresponsive drive motors where two confirmed software bugs were fixed but the root cause remains at the hardware layer. As a direct consequence, Tasks B and A were fully designed at the architectural level but could not be implemented or tested on hardware.

This report presents the working hypothesis for the remaining motor fault, a documented bring-up path for Orin-generation JetBot hardware, and the complete design for the fused-perception and driving-control pipelines.

Keywords: Jetson Orin Nano, JETANK, Autonomous Driving, Camera Fusion, Edge AI, Robotics

Table of Contents

Declaration of honour	ii
Abstract	iii
Table of Contents	iv
List of Tables	vi
1 Introduction	1
1.1 Why Edge AI Robotics	1
1.2 Why Camera Fusion	1
1.3 The JetBot / Orin Generation Gap	1
1.4 Task Scope	2
2 Materials and System Overview	3
2.1 Software Stack	3
3 Methodology	5
3.1 March — Platform Misidentification and OS Selection	5
3.2 April — WiFi, Repository Research, and Power Discovery	5
3.3 Late April — WiFi Resolution and Python Environment	6
3.4 May — Desktop Environment and System Stabilisation	6
3.5 Hardware Bring-Up — Camera, OLED, and Servos	6
3.6 Drive Motor Investigation (Ongoing at Time of Reporting)	6
4 Results: Task C — Hardware Assembly and Integration	7
4.1 Platform Identification and OS Flashing	7
4.2 Desktop Environment (GDM3)	7
4.3 CSI Camera (IMX219-160)	9
4.4 OLED Display	9
4.5 Power Architecture Mismatch	9
4.6 Servo Calibration	9
4.7 Drive Motor Bring-Up	10

5 Results: Task B — Camera Fusion for Perception	11
5.1 Fused State Schema	11
5.2 Planned Robustness Evaluation	12
5.3 Status	12
6 Results: Task A — Camera Fusion for Driving	13
6.1 Controller Policy (Designed)	13
6.2 Safety Gates (Designed)	13
6.3 Closed-Loop Evaluation Plan (Designed)	14
6.4 Status	14
7 Discussion of Critical Blockers	15
7.1 Blocker 1 — Power Architecture	15
7.2 Blocker 2 — Drive Motors Unresponsive	15
7.3 Downstream Effect on Tasks B and A	16
8 Conclusion	17
9 Recommendations	18
9.1 To Close Out the Drive Motor Issue	18
9.2 To Close Out the Power Architecture	18
9.3 For Future Orin-Generation JetBot / JETANK Projects	19
9.4 Once Tasks B and A Are Unblocked	19
10 Self-Reflection Note	20
Bibliography	22
Appendix A Full Resolved-Issue Log	23

List of Tables

Table 2.1	Hardware components used in the project.	3
Table 2.2	Software stack.	4
Table 4.1	Summary of ten resolved issues in Task C.	8
Table 10.1	Individual contribution percentages.	20
Table A.1	Complete issue log with blocking status.	23

Chapter 1

INTRODUCTION

1.1 Why Edge AI Robotics

Autonomous mobile robots that perform inference locally, rather than relying on a cloud connection, are essential wherever latency, privacy, or connectivity cannot be guaranteed. Warehouses, hospitals, agricultural inspection, and last-mile delivery are common use cases. The NVIDIA Jetson family of single-board computers provides GPU-accelerated edge inference in a form factor compact enough for small robots, making it a natural teaching platform for this kind of system [1, 3].

1.2 Why Camera Fusion

Individual perception modalities are fragile in isolation. Lane detection alone fails the moment an obstacle obscures the road markings; object detection alone cannot determine where the robot sits within its lane. Fusing both into a single per-frame state (lane offset plus obstacle class, position, and distance) produces a more complete and robust basis for control decisions than either signal alone. This is the central technical idea the project set out to demonstrate.

1.3 The JetBot / Orin Generation Gap

The widely used NVIDIA JetBot reference platform targets the original Jetson Nano (2019, JetPack 4.x). This project instead used the newer Jetson Orin Nano Super (2024, JetPack 6.x), mounted on a Waveshare JETANK chassis that was itself designed around the older Nano [10]. The two Jetson generations differ substantially in power requirements, kernel packaging, camera driver model, and peripheral addressing. Existing JetBot and

JETANK tutorials assume the older hardware and require substantial adaptation at nearly every layer of the stack. A secondary contribution of this project, beyond the three assigned tasks, is a documented bring-up path for teams attempting the same Orin-generation upgrade in the future.

1.4 Task Scope

Task C — Hardware Assembly and Platform Integration Chassis build, compute module mounting, camera/motor/display validation, and remote development workflow.

Task B — Camera Fusion for Perception Combine a lane-following model and a YOLO-based object detector [6, 9] into one structured, per-frame perception state.

Task A — Camera Fusion for Driving Convert the fused perception state into a safety-gated control policy (steering correction, speed control, emergency stop).

Chapter 2

MATERIALS AND SYSTEM OVERVIEW

This chapter lists the hardware and software configuration as of the end of the semester, including the corrected channel mappings and bus numbers discovered during bring-up (Chapter 4).

Table 2.1: Hardware components used in the project.

Component	Model / Spec	Purpose
Main compute	NVIDIA Jetson Orin Nano Super (8 GB)	GPU-accelerated inference; 6× ARM Cortex-A78, Ampere GPU, 1024 CUDA cores
Chassis	Waveshare JETANK AI Kit	Tank-track chassis; TB6612FNG dual H-bridge motor driver [8]; PCA9685 PWM controller [5]
Camera	Waveshare IMX219-160 (CSI)	160° FOV, 8 MP, MIPI CSI-2 to CAM0
Servos	Feetech SCS15-AP ×5	Serial bus servos for 5-DOF arm (2 mounted: base rotation + camera tilt)
Display	SSD1306 OLED (128×32)	System status: IP address, CPU/RAM, component health
Power (Jetson)	Veger T100 100 W USB-C PD bank + PD trigger cable (15 V, 5.5×2.5 mm barrel)	Independent 9–20 V supply for Orin mainboard
Power (motors)	18650 cells (3S, ~12.6 V) via JETANK expansion board	Motor/servo rail only
Sensors	INA219 current sensor	Battery voltage/current monitoring
Networking	Intel Wireless 8265/8275 (PCIe)	WiFi via wlp1p1s0 (requires DKMS backport on JetPack 6.x)

2.1 Software Stack

Table 2.2: Software stack.

Component	Version / Detail
OS	JetPack 6.2 (L4T R36.4.4, kernel 5.15.148-tegra)
GPU compute	CUDA 12.6, cuDNN 9.3.0.75, TensorRT 10.3.0.30 [4]
ML framework	PyTorch via Docker containers (moatazsawi/jetbot-orin) [7]
Computer vision	OpenCV 4.8.0, GStreamer with <code>nvarguscamerasrc</code>
Motor control	PCA9685 driver (<code>smbus2</code>), TB6612FNG via custom <code>Robot</code> class
Servo control	Feetech SCS SDK (<code>SCSCTRL</code>), 1 Mbps on <code>/dev/ttyTHS1</code>
Display	<code>luma.oled</code> (SSD1306, I2C bus 7), <code>systemd</code> service
Container runtime	Docker 28.2.2

Chapter 3

METHODOLOGY AND PROJECT TIMELINE

The project proceeded as a series of bring-up sessions, each resolving one or more blocking issues before the next layer of the stack could be attempted. This chapter summarises the chronological path; full session notes are retained in the project archive.

3.1 March — Platform Misidentification and OS Selection

Early sessions assumed the hardware was a Jetson Nano 4 GB and attempted to flash JetPack 4.x/5.x images accordingly. All such images failed to boot. The team subsequently identified the actual hardware as a Jetson Orin Nano Super (module P3767, carrier board P3768) and determined that JetPack 6.0+ is required for this SoC generation. The earlier JetPack lines target the Tegra X1 architecture used in the original Nano, not the Orin's Tegra234/Ampere architecture. The JetPack 6.2 SD card image was the first to boot successfully.

3.2 April — WiFi, Repository Research, and Power Discovery

With SSH access established over Ethernet, the team diagnosed the WiFi blocker: the onboard Intel Wireless 8265/8275 card was physically present on the PCIe bus but produced no network interface because JetPack 6.x ships a stripped-down kernel without the `iwlwifi` module. The team installed the `backport-iwlwifi-dkms` package to resolve this. During hardware reassembly, the team discovered that the JETANK expansion board cannot power the Jetson Orin Nano mainboard at all, which fundamentally reshaped the project's power architecture (Section [4.5](#)).

3.3 Late April — WiFi Resolution and Python Environment

The DKMS backport was explicitly installed against the running kernel, bringing up the `wlP1p1s0` interface permanently. In parallel, the team resolved a broken apt-managed pip installation by bootstrapping pip directly from the PyPA install script, after a virtual-environment workaround proved insufficient for system-wide package management.

3.4 May — Desktop Environment and System Stabilisation

A silent GDM3 failure (the `gdm3` binary itself was missing from the JetPack image, masked by a `systemd RemainAfterExit` flag) was diagnosed and fixed by reinstalling the package. With WiFi, pip, and the desktop environment all stable, the team turned to full hardware assembly: camera, OLED, servos, and finally drive motors.

3.5 Hardware Bring-Up — Camera, OLED, and Servos

The CSI camera, OLED status display, and serial bus servos were each brought online during this phase, each requiring a JetPack 6.2-specific fix (device-tree overlay, I2C bus renumbering, and calibration-value correction respectively). These are detailed fully in [Chapter 4](#).

3.6 Drive Motor Investigation (Ongoing at Time of Reporting)

The final and most significant blocker was the drive motor system: all motion commands executed without error but produced no physical movement. Two confirmed software bugs were fixed ([Section 4.7](#)), but the root cause was not fully resolved before the reporting deadline, which in turn blocked implementation of Tasks B and A.

Chapter 4

RESULTS: TASK C — HARDWARE ASSEMBLY AND INTEGRATION

Task C covered chassis assembly, Jetson platform integration, and validation of every onboard subsystem (camera, display, servos, motors) plus a stable remote development workflow. Ten distinct system-level issues were identified and resolved over the course of the semester; they are summarised in Table 4.1 and detailed in the following sections.

4.1 Platform Identification and OS Flashing

The single highest-leverage finding of the semester was platform identification: the Jetson Orin Nano Super requires JetPack 6.0 or later [2]. Several legacy SD card images were tried and all failed to boot: `jetbot-043_nano-4gb-jp45`, `jetson-nano-jp461-sd-card-image`, `jp60dp-orin-nano-sd-card-image`, and `JP512-orin-nano-sd-card-image`; only the JetPack 6.2 SD card image succeeded. Flashing required Balena Etcher rather than Rufus, since Rufus cannot write raw Linux partition images, and a Diskpart clean of the SD card on Windows before flashing.

4.2 Desktop Environment (GDM3)

A particularly deceptive failure: `systemctl status` reported the display manager as “active (exited)”; normally a red flag would be more obvious. The root cause was that the `gdm3` binary itself had never been installed in the JetPack image; the init script’s executable check silently exits when the binary is missing, and a `RemainAfterExit=yes` flag in the `systemd` unit made the service appear to have run successfully. Reinstalling the package resolved it.

Table 4.1: Summary of ten resolved issues in Task C.

Category	Problem	Root Cause	Fix
OS	Image would not boot	JetPack 4.x/5.x images target Tegra X1 (Nano), not Tegra234/Ampere (Orin)	Flashed JetPack 6.2 SD card image
Networking	No WiFi interface present	iwlwifi kernel module stripped from JetPack 6.x kernel	Installed backport-iwlwifi-dkms; explicit dkms install step against running kernel
Desktop	No GUI on boot (GDM3 “active (exited)”))	/usr/sbin/gdm3 binary missing from JetPack image; RemainAfterExit masked failure	apt install -reinstall gdm3
Python	pip broken via apt	Version conflict between python3-setuptools and python3-pkg-resources	Bootstrapped pip via PyPA install script (bypassing apt)
Camera	No /dev/video0 device	IMX219 driver activated via device-tree overlay on JetPack 6.2, not a loadable module	Added imx219-A.dtbo overlay to extlinux.conf
Display	OLED stayed blank	Hardcoded I2C bus 1, hardcoded wlan0 interface, unavailable libraries	Rewrote stats.py using luma.oled, I2C bus 7, wlp1p1s0; deployed as systemd service
Servos	Aggressive spin, cable tangling	Centre positions drifted from factory default of 512 after mounting	Read back centres via serial scan; updated servoInit in installed package and source
Servos	Calibration script crashed silently	servoInt.py used time.sleep() without import time	Added the missing import
Motors	PCA9685 channel forced HIGH	set_pwm(channel, 0, 0) sets ON=OFF=0; ON has priority (brake mode)	Changed to set_pwm(channel, 0, 4096) for correct FULL_OFF
Motors	Wrong PCA9685 channel assignment	motor.py used Adafruit MotorHAT layout instead of Waveshare JETANK layout	Remapped to Waveshare standard (PWMA=0/AIN1=1/AIN2=2, PWMB=5/BIN1=3/BIN2=4)

4.3 CSI Camera (IMX219-160)

Unlike most kernel drivers, the IMX219 sensor driver on JetPack 6.2 is enabled via a device-tree overlay rather than a loadable kernel module. Without an `OVERLAYS` line referencing the correct `.dtbo` file in `/boot/extlinux/extlinux.conf`, the video input hardware initialises but finds no configured sensor, and no `/dev/video*` device is created. Adding the `imx219-A` overlay (for CAM0) resolved this; the `-C` variant is reserved for CAM1 if the ribbon cable is ever moved.

4.4 OLED Display

The original `JetBot stats.py` script depended on three assumptions that do not hold on this hardware: I2C bus 1 (the JETANK expansion board uses bus 7), the `wlan0` interface name, and the `qwiic` / `Adafruit_SSD1306` libraries (not installed on JetPack 6.2 and effectively unmaintained). The script was rewritten using `luma.oled` and `psutil`, and deployed as a `systemd` service so the display comes up automatically on every boot.

4.5 Power Architecture Mismatch

The JETANK expansion board was designed for the original Jetson Nano, which accepts 5 V via its GPIO header. The Jetson Orin Nano Super requires 9–20 V delivered through a DC barrel jack and cannot be powered through GPIO at all. This meant the two boards had to be powered through entirely separate paths: a Veger T100 USB-C Power Delivery bank (set to 15 V via a PD trigger cable into a 5.5×2.5 mm barrel plug) for the Jetson, and the existing 18650 battery pack for the motor/servo rail through the expansion board. The two rails share only a common ground through the GPIO header, which is sufficient for I2C and serial communication.

One behavioural quirk: GPIO pins 2 and 4 are always live at 5 V whenever the Jetson itself is powered, regardless of the expansion board's switch state, because the OLED, INA219, and PCA9685 are all powered from the Jetson side, not the battery side.

This power solution was identified and specified in detail but had not been procured and tested end-to-end before the reporting deadline. This is recorded as Blocker 1 in Chapter 7.

4.6 Servo Calibration

Feetech SCS15-AP serial bus servos (IDs 1–5, daisy-chained on `/dev/ttyTHS1`) initially spun aggressively to incorrect positions on first command, tangling the base-rotation cable around the chassis. The cause was that the servos' true centre positions had drifted from the factory default of 512 after physical mounting. Reading back the actual centre positions

via a serial bus scan and updating `servoInit` to `[None, 1021, 512, 512, 512, 811]` in both the installed package and the source repository resolved the issue.

A second, unrelated bug (a missing `import time` in the calibration script) was also fixed. Servo 1 (base rotation) has no physical end-stop and is now constrained in software to $\pm 80^\circ$ from centre to prevent future cable tangling.

4.7 Drive Motor Bring-Up

Two confirmed bugs were found and fixed in the `jetbot_waveshare` Python package:

1. **PCA9685 brake-mode bug:** the driver used `set_pwm(channel, 0, 0)` to represent a fully-off PWM value, but per the PCA9685 datasheet, when `ON=OFF=0` the `ON` bit takes priority and the pin is forced permanently HIGH, keeping the TB6612FNG's direction pin high regardless of the commanded duty cycle. This was corrected to `set_pwm(channel, 0, 4096)`, the documented `FULL_OFF` encoding.
2. **Incorrect channel map:** the code used the Adafruit MotorHAT layout (channels 2–4 for motor 1, 5–7 for motor 2) inherited from the original JetBot codebase, instead of the Waveshare JETANK's actual layout (`PWMA=0/AIN1=1/AIN2=2` for motor 1, `PWMB=5/BIN1=3/BIN2=4` for motor 2).

Neither fix produced motor movement. This is recorded as Blocker 2 and discussed in full in [Chapter 7](#).

Chapter 5

RESULTS: TASK B — CAMERA FUSION FOR PERCEPTION

Task B's objective was to produce a single fused perception state per frame, combining a lane-following baseline (from the `road_following` reference notebooks) with an object-detection baseline (YOLO, from the `object_following` reference notebooks). The fusion design was completed at the architectural level; implementation and on-hardware testing were blocked by the unresolved power and motor issues described in Chapters 4 and 7.

5.1 Fused State Schema

The designed per-frame fusion output combines a continuous lane-offset/confidence signal with a discrete obstacle detection, plus a derived risk level and an action hint intended for direct consumption by the Task A controller:

```
{
  "lane_offset": 0.10,
  "lane_conf": 0.84,
  "obstacle": {"class": "person", "conf": 0.92, "distance_m": 1.7},
  "risk_level": "high",
  "action_hint": "stop"
}
```

Listing 5.1: Fused perception state schema

5.2 Planned Robustness Evaluation

The evaluation plan called for testing the fused pipeline under five degraded conditions, with frame-by-frame logging of FPS, confidence stability, and qualitative failure cases for each:

- Lighting change
- Shadows and glare
- Motion blur
- Partial occlusion of the lane or the obstacle
- Different camera angles and resolutions

5.3 Status

Not started on hardware. The lane-following and object-detection reference notebooks (`road_following`, `object_following`) were identified and reviewed as the intended base-lines, and the Docker images for the ML and display containers were pulled and verified runnable, but live fusion testing requires a driveable robot, which was not achieved this semester.

Chapter 6

RESULTS: TASK A — CAMERA FUSION FOR DRIVING

Task A's objective was to convert the Task B fused perception state into safe, real-time driving commands. As with Task B, the controller logic was fully designed but not implemented or tested on hardware.

6.1 Controller Policy (Designed)

- Lane centred and clear → move forward
- Lane offset detected → apply steering correction proportional to the offset
- High risk level or low perception confidence → slow down or stop

6.2 Safety Gates (Designed)

- **Confidence-threshold stop:** if lane or obstacle confidence falls below a set threshold, halt rather than act on an uncertain perception state
- **Obstacle-distance-threshold stop:** halt if a detected obstacle is closer than a safe following distance
- **Manual emergency-stop path:** an independent, software-prioritised stop command that overrides the controller output

6.3 Closed-Loop Evaluation Plan (Designed)

Three scenarios were planned for quantitative and qualitative evaluation once the robot could drive:

1. Normal lane following
2. An obstacle appearing in the robot's path
3. An ambiguous, low-confidence perception case

For each scenario, the plan calls for tracking intervention count, command stability/jitter, and safe-stop trigger reliability.

6.4 Status

Not started on hardware. This task is fully gated on Task B perception output and a working drive-motor system (Chapter [7](#), Blocker 2).

7

Chapter

DISCUSSION OF CRITICAL BLOCKERS

7.1 Blocker 1 — Power Architecture

The JETANK expansion board cannot power the Jetson Orin Nano under any configuration; this is a fixed property of the hardware, not a bug. A specific solution (a Veger T100 100 W USB-C PD power bank with a 15 V trigger cable into the Orin's barrel jack) was identified, with the constraint that the connection must be USB-C to USB-C (a USB-A source bypasses PD negotiation entirely and the bank falls back to 5 V, which will not boot the Jetson). This was not procured and tested in time to support full untethered operation.

7.2 Blocker 2 — Drive Motors Unresponsive

This is the project's most significant open technical problem. After fixing the two confirmed software bugs (Section 4.7), the team ruled out the following as causes through direct testing:

- PCA9685 not present on the I2C bus: ruled out; confirmed at address 0x60 on bus 7 via `i2cdetect`
- Insufficient battery voltage: ruled out; INA219 reported 11.85 V (79% charge) during testing
- Incorrect motor wiring to the expansion board terminals: ruled out; connections were physically verified
- Wrong Python import path: ruled out; `from jetbot import Robot` was confirmed to resolve to the corrected `jetbot_waveshare` package
- I2C communication failures: ruled out; no I2C errors were thrown on any `set_pwm` call

An exhaustive brute-force scan of plausible PCA9685 channel-to-motor mappings, including STBY-candidate channels 8, 9, 12, 13, 14, and 15, also produced no movement on any combination.

What remains unverified:

- Whether PCA9685 register writes are actually sticking (no register read-back has been performed)
- The exact PCA9685-to-TB6612FNG pin wiring on the JETANK expansion board (no official schematic is publicly available)
- Whether the TB6612FNG's STBY pin is hardwired high or requires explicit activation from an as-yet-untried channel
- Whether the TB6612FNG's VM pin is actually receiving the 12 V motor-supply rail through the expansion board's traces

The working hypothesis is that the fault sits below the software layer the team controls, most likely a wiring or enable-pin issue on the expansion board itself, only resolvable through direct multimeter probing or a manufacturer-provided schematic.

7.3 Downstream Effect on Tasks B and A

Because Tasks B and A both assume a robot capable of driving, neither could move past the design stage this semester. The perception-fusion schema (Section 5.1) and the safety-gated controller policy (Sections 6.1–6.2) represent the team's complete intended design and are ready to be implemented as soon as Blockers 1 and 2 are resolved.

Chapter 8

CONCLUSION

This project set out to build an autonomous, camera-fusion-driven JetBot on a Jetson Orin Nano Super. The headline technical result is not the driving demo originally scoped, but a thorough, hard-won account of why an Orin-generation Jetson is not a drop-in replacement for the Jetson Nano the JETANK kit and most public JetBot tutorials were designed around, and a documented, ten-issue bring-up path that resolves every layer of that mismatch except the motor driver hardware itself.

Ten distinct system-level issues were diagnosed and fixed across OS imaging, kernel driver support, desktop environment, Python packaging, CSI camera enablement, OLED display, and serial servo calibration. Two blockers remain open: an unprocured power-bank solution for the Jetson mainboard, and an unresolved drive-motor fault that has been narrowed to the hardware layer through a careful process of elimination. The perception-fusion and driving-control logic for Tasks B and A were fully designed and are implementation-ready.

Although the robot did not drive by the end of the semester, the project produced a complete and transferable diagnostic record: every fix is traceable to a specific root cause, and every open item has a specific, actionable next diagnostic step (Chapter 9). That record is, itself, the project's primary deliverable.

Chapter 9

RECOMMENDATIONS AND FUTURE WORK

9.1 To Close Out the Drive Motor Issue

1. Read back PCA9685 LED0–15 registers via `i2cget` after each write to confirm values are actually sticking in hardware, ruling in or out a silent I2C write failure.
2. Probe the TB6612FNG directly with a multimeter while motion commands are issued: VM (motor supply voltage), STBY (should be above 2 V for the H-bridge to be enabled), and IN1/IN2 (should toggle between 0 V and 3.3 V with commanded direction).
3. Expand the channel brute-force search to include PCA9685 channels 8–15 in every combination tried, not only the previously assumed STBY candidates.
4. Temporarily wire one motor directly to the battery pack to rule out a damaged motor before continuing to debug the driver chain.
5. Contact Waveshare support directly for the JETANK expansion board schematic, or at minimum the PCA9685-to-TB6612FNG pin mapping.

9.2 To Close Out the Power Architecture

1. Procure the Veger T100 and the 15 V PD trigger cable (5.5×2.5 mm barrel) and test the full wireless-plus-battery configuration end-to-end.
2. Evaluate the Waveshare UPS Module (C), built specifically for the Jetson Orin family around 21700 cells, as a cleaner single-package alternative.
3. Mount the chosen power source low and centred on the chassis for balance once finalised.

9.3 For Future Orin-Generation JetBot / JETANK Projects

1. Use JetPack 6.2 (or later) as the baseline image from day one; earlier JetPack lines are incompatible with Orin hardware at the SoC level.
2. Add the IMX219 device-tree overlay and install `backport-iwlwifi-dkms` as standard first-boot steps immediately after flashing.
3. Replace every `wlan0` reference in inherited scripts with `wlp1p1s0`; JetPack 6.x uses predictable network interface naming.
4. Confirm the I2C bus number for the expansion board before writing any peripheral code; it is bus 7 on this hardware, not the bus 1 most tutorials assume.
5. Bootstrap pip directly from the PyPA install script rather than relying on apt-managed pip, which is broken out of the box on JetPack 6.x.
6. Plan for an NVMe SSD upgrade early; the 28 GiB microSD card fills quickly once Docker images and notebooks are added (83% used before any ML datasets were stored).

9.4 Once Tasks B and A Are Unblocked

1. Implement and test the Task B fusion pipeline against the planned robustness scenarios (lighting, shadows/glare, blur, occlusion, camera angle/resolution changes) before integrating with Task A.
2. Implement the Task A controller exactly as designed in Chapter 6, then run the three planned closed-loop scenarios and report intervention count, command stability, and safe-stop reliability as the project's quantitative result set.

Chapter 10

SELF-REFLECTION NOTE

Individual Contribution Breakdown

Table 10.1: Individual contribution percentages.

Team Member	Contribution (%)
Adam Ryan	[%]
Aidan Ryan	[%]
Denis Vasilev	[%]
Nora Regina Cervený	[%]
Total	100%

Adam Ryan

[Write your self-reflection here — approximately 8–10 lines. What did you learn? What challenges did you face? How did you contribute to the team? What would you do differently?]

Aidan Ryan

[Write your self-reflection here — approximately 8–10 lines. What did you learn? What challenges did you face? How did you contribute to the team? What would you do differently?]

Denis Vasilev

[Write your self-reflection here — approximately 8–10 lines. What did you learn? What challenges did you face? How did you contribute to the team? What would you do differently?]

Nora Regina Cervený

[Write your self-reflection here — approximately 8–10 lines. What did you learn? What challenges did you face? How did you contribute to the team? What would you do differently?]

BIBLIOGRAPHY

- [1] NVIDIA Corporation, “Jetbot: An open-source robot based on nvidia jetson nano,” 2019. [Online]. Available: <https://github.com/NVIDIA-AI-IOT/jetbot>
- [2] —, “Jetpack sdk documentation,” 2024. [Online]. Available: <https://developer.nvidia.com/embedded/jetpack>
- [3] —, “Jetson orin nano series technical brief,” 2024. [Online]. Available: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>
- [4] —, “Tensorrt: Sdk for high-performance deep learning inference,” 2024. [Online]. Available: <https://developer.nvidia.com/tensorrt>
- [5] NXP Semiconductors, “Pca9685: 16-channel, 12-bit pwm fm+ i2c-bus led controller,” 2023. [Online]. Available: <https://www.nxp.com/docs/en/data-sheet/PCA9685.pdf>
- [6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.
- [7] M. Sawi, “Jetbot orin: Jetbot adapted for jetson orin,” 2024. [Online]. Available: <https://github.com/moatazsawi/jetbot-orin>
- [8] Toshiba Semiconductor, “Tb6612fng: Dual motor driver datasheet,” 2015. [Online]. Available: https://toshiba.semicon-storage.com/info/TB6612FNG_datasheet_en_20151009.pdf
- [9] Ultralytics, “Ultralytics yolo: Real-time object detection and image segmentation,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [10] Waveshare Electronics, “Jetank ai kit wiki,” 2024. [Online]. Available: https://www.waveshare.com/wiki/JETANK_AI_Kit

Appendix A

FULL RESOLVED-ISSUE LOG

This appendix expands Table 4.1 with the blocking status of each issue, for archival and grading reference.

Table A.1: Complete issue log with blocking status.

#	Issue	Resolution	Blocker for Driving?
1	Wrong OS for Orin hardware (JP4.x/5.x images)	JetPack 6.2 SD card image	Yes — resolved
2	No WiFi (iwlwifi missing from kernel)	backport-iwlwifi-dkms	Yes — resolved
3	GDM3 binary missing	apt install -reinstall gdm3	No
4	pip broken via apt	Bootstrapped pip via PyPA install script	No
5	Camera not detected (no /dev/video0)	IMX219 device-tree overlay in extlinux.conf	No
6	OLED: wrong I2C bus, interface name, missing libs	Rewrote with luma.oled, bus 7, w1P1p1s0	No
7	Servo centres drifted from factory default	Updated servoInit in both TTLServo.py copies	No
8	servoInt.py missing import time	Added the missing import	No
9	PCA9685 set_pwm(0,0) forces pin HIGH (brake mode)	Changed to set_pwm(0, 4096) for FULL_OFF	Yes — critical (fix applied, motors still not moving)

Table A.1 — continued from previous page

#	Issue	Resolution	Blocker for Driving?
10	Motor channel mapping wrong for Waveshare hardware	Updated to Waveshare PCA9685 channel layout	Yes — critical (fix applied, motors still not moving)